

Project Guide: Build Your Own Teachable Machine

A Step-by-Step Practical Blueprint for Bootcamp Students

1. Introduction to the Project

Welcome to your final full-stack AI development project! In this assignment, you are going to replicate the core experience of **Google's Teachable Machine**, but with a major professional upgrade: you will build a completely decoupled, client-server system using **Streamlit** for your User Interface and **FastAPI** for your machine learning backend.

Instead of running ML scripts locally inside a Jupyter Notebook, you will build an application that accepts dynamic image inputs, processes them across a web network, trains a model on the fly based on custom user labels, and delivers instant, real-time predictions.

2. Reference & Benchmark for Testing

Before writing a single line of code, you must experience the desired workflow firsthand. Use the official Google platform to test how rapid image gathering, instant local training, and live performance bar charts work:

Explore the Baseline UI/UX Reference Here:

<https://teachablemachine.withgoogle.com/train/image>

Your Goal: Your Streamlit web interface should mimic this exact user workflow: (1) Define custom category names, (2) Collect dataset inputs via files or webcam, (3) Click a "Train" button, and (4) Instantly test the accuracy with a live preview meter.

3. The Architecture Explained Simply

Your application is split into two independent services that communicate over standard web channels:

- **The Frontend (Streamlit):** This is the visual layout. It handles the web camera frame grabs, processes file drag-and-drops, displays action buttons, and draws the output charts. It *does not* do any machine learning calculations. It simply collects data and sends it away using HTTP web requests.
- **The Backend (FastAPI):** This is the brain of your application. It acts as an active web server. It receives images sent by Streamlit, saves them inside dedicated folders on your hard drive, runs the training algorithms when prompted, and generates prediction metrics.

Why split them up? This is exactly how commercial software is built! By decoupling the user interface from the heavy backend computations, your frontend stays lightweight and highly interactive, while your backend can be moved to powerful cloud servers equipped with high-performance hardware seamlessly.

4. Your Development Milestones

Follow this sequence to prevent feeling overwhelmed and to build a robust codebase step by step:

MILESTONE 1 Setup the FastAPI Workspace & Image Ingestion

Create your root backend folders. Write a POST endpoint called `/upload-sample` that accepts a text string named `class_name` and a list of image files. Use Python's built-in `os` module to check if a folder named after that class exists inside your dataset directory. If it doesn't, create it automatically. Save incoming images with completely randomized filenames (hint: use the `uuid` module) to prevent file overwriting.

MILESTONE 2 Build the Transfer Learning Training Engine

Implement a POST endpoint called `/train`. When this endpoint is triggered, your backend must scan the dataset directory, read all available class sub-folders, and load images. Instead of training a model from scratch, use a pre-trained backbone model like **MobileNetV3** from PyTorch to extract quick image features. Train a fast classifier (like a Scikit-Learn LogisticRegression model) on top of these extracted features. Once complete, save the trained model weights as a local file (e.g., `model.pkl`).

MILESTONE 3 Build the Inference & Prediction API

Write a POST endpoint called `/predict` that accepts a single image. This endpoint should load your saved `model.pkl` weights, parse the incoming image through the exact same PyTorch preprocessing transformations, extract its structural features, and output the target category along with its percentage probability score.

MILESTONE 4 Connect the Streamlit User Interface

Now, build your user interface! Create a text field for entering class names and provide a file uploader or use `st.camera_input` to capture frames. Use the Python `requests` library to send data directly to your FastAPI endpoints. Use `st.button` to trigger the training loop, and display final prediction scores visually using clean elements like `st.progress()` or `st.bar_chart()`.

5. Key Architectural Rules to Remember

1. **Pre-processing Uniformity:** Machine Learning models are highly sensitive to dimensions. An image passed during your live testing phase must be transformed using the exact same steps applied during the

training phase (e.g., resized precisely to 224x224 pixels and normalized according to default channel parameters).

2. **State Management Checkpoints:** Your Streamlit dashboard should look clean. Do not show testing utilities or prediction components until the user has successfully run a training execution cycle. Track this state safely using `st.session_state`.
3. **Graceful Error Controls:** Your application must be stable. If a user accidentally presses the "Train Model" button before uploading images, or with only a single class folder populated, your FastAPI server must catch this error smoothly and return a clear, readable warning message instead of crashing entirely.

Final Challenge Tip: When you wrap up your local features, try packing your applications into separate Dockerfiles and launching them seamlessly using a single `docker-compose.yml` file to achieve an industry-grade software configuration.